CLOUD & PLATFORMS

Azure AI Services

Azure OpenAI models & deployments, Cognitive Services (Vision, Language, Speech, Document Intelligence), AI Search vector & hybrid RAG, AI Foundry, Responsible AI, Python SDK patterns, cost & quotas — complete interview-prep guide

azure · openai · ai-services · cognitive · rag · responsible-ai · interview-prep

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. Azure AI Services — Overview & Taxonomy
2. Azure OpenAI Service — Models & Deployments
3. Chat Completions, Embeddings & Streaming API
4. Azure AI Vision & Computer Vision
5. Azure AI Language Services
6. Azure AI Speech Services
7. Azure AI Document Intelligence
8. Azure AI Search — Vector & Hybrid Search
9. RAG Pattern End-to-End
10. Azure AI Foundry & ML Studio
11. Responsible AI & Content Safety
12. Authentication: Keys, Endpoints & RBAC
13. Python SDK Integration Patterns
14. Cost, Quotas & Azure OpenAI vs. Direct OpenAI API
15. Best Practices & Interview Cheat Sheet

Azure AI Services — Overview & Taxonomy

Azure AI Services is Microsoft's unified portfolio of production-ready AI APIs, formerly known as Azure Cognitive Services. Since 2023 the umbrella explicitly includes Azure OpenAI Service, Azure AI Search, and Azure AI Foundry alongside the classic Vision,

Language, Speech, Document Intelligence, and Content Safety workloads — all accessed through a single Azure resource type: `Microsoft.CognitiveServices/accounts`.

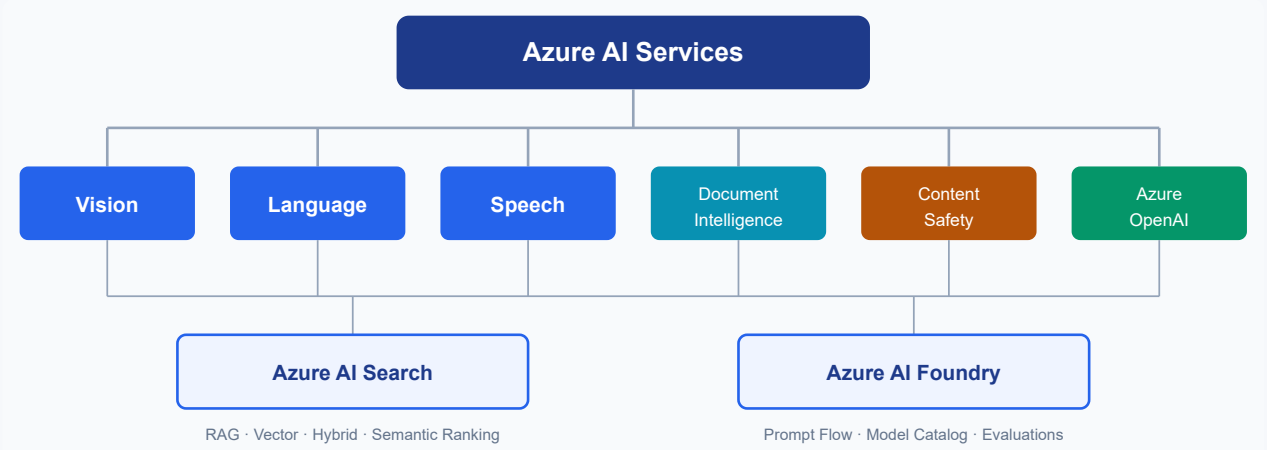


Figure 1 — Azure AI Services platform taxonomy. Vision, Language, Speech, Document Intelligence, Content Safety and Azure OpenAI are the six leaf services; AI Search and AI Foundry are cross-cutting platform components.

Aspect	Detail
Resource type	`Microsoft.CognitiveServices/accounts` (single resource for all services, or per-service dedicated resources)
Auth methods	API key header `api-key` / `Ocp-Apim-Subscription-Key`, or Microsoft Entra ID bearer token
Endpoint pattern	`https://<resource-name>.cognitiveservices.azure.com/`
Multi-service resource	One key + endpoint routes to all services; per-service resources give finer quota control
Pricing tiers	Free (F0, heavily throttled), Standard (S0+, pay-as-you-go), Commitment tiers for predictable costs
Data privacy	Data is NOT used to train Microsoft models; stays in your Azure region; subject to your tenant's compliance

> **Branding note**
> "Cognitive Services" is the legacy name. Since 2023 Microsoft brands everything as **Azure AI Services** in the portal and docs. The ARM resource type and many SDK package names still say *cognitiveservices*.

Azure OpenAI Service — Models & Deployments

Azure OpenAI Service is the enterprise-grade access layer for OpenAI's models. Rather than calling api.openai.com directly, you call your own Azure-hosted endpoint with your data remaining in your tenant. You provision **deployments** — named instances of a specific model version — and call them by deployment name, not model ID.

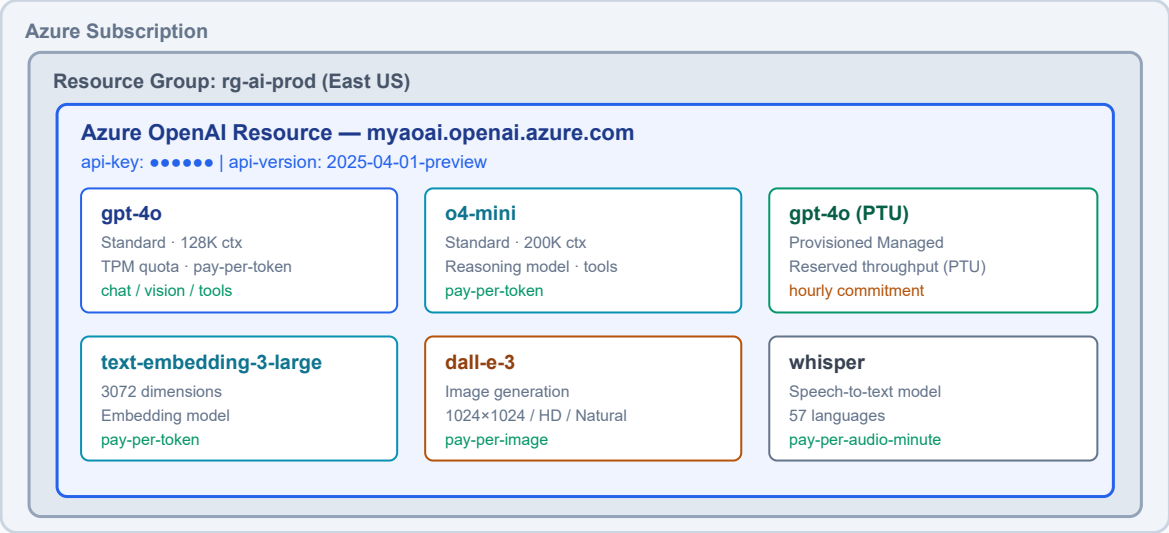


Figure 2 — One Azure OpenAI resource hosts multiple named deployments. Each deployment targets one model version and has its own quota and pricing model.

Model Families Cheat Sheet

Model Family	Key Models (2025-2026)	Best For	Context
GPT-4o	gpt-4o, gpt-4o-2024-11-20	Multimodal chat, vision, function calling	128K
GPT-4.1	gpt-4.1, gpt-4.1-mini	Long-context instruction following, coding	1M (4.1)
o-Series	o3, o4-mini	Complex reasoning, math, code generation	200K
GPT-4o-mini	gpt-4o-mini	Fast, cost-effective tasks, classification	128K
Embeddings	text-embedding-3-large, text-embedding-3-small, ada-002	Semantic search, RAG indexing	8K input
DALL-E	dall-e-3	Image generation from text prompt	4K prompt
Whisper	whisper	Audio transcription and translation	25 MB / file

Deployment Types

Type	Capacity	Billing	Best For
Standard	Shared, may throttle at quota	Pay-per-token	Dev, variable workloads
Global Standard	Microsoft-routed globally	Pay-per-token	High availability, burst traffic
Provisioned Managed (PTU)	Reserved throughput units	Hourly commitment fee	Latency-sensitive production
Batch	Async processing queue	50 % discount vs Standard	Bulk inference, offline jobs

✦ **PTU vs Standard**

Provisioned Throughput Units (PTU) give deterministic, low-latency responses because capacity is reserved for you. Break-even vs Standard typically occurs around 60–70 % sustained utilization. Below that, Standard is cheaper.

What is the difference between a model name and a deployment name in Azure OpenAI?

The **model name** (e.g., `gpt-4o`) identifies the AI architecture and version in the catalog. The **deployment name** is a custom label you assign when you create a deployment in your resource (e.g., `my-gpt4o-prod`). In API calls you always reference the deployment name in the `model` field — Azure resolves it to the underlying model. This lets you swap model versions by redeploying without changing application code.

When should you choose Provisioned Managed (PTU) over Standard deployment?

Choose PTU when you need **guaranteed, predictable latency** and your workload has steady, high throughput (typically above 60–70% sustained utilization). Standard is better for variable or bursty traffic because you only pay for what you use. PTU requires an upfront capacity reservation (billed per hour per PTU) so it is cost-effective only when utilization is consistently high.

Chat Completions, Embeddings & Streaming API

Azure OpenAI exposes the same Chat Completions, Completions, Embeddings, and Images APIs as OpenAI — with the addition of an `azure_endpoint` and an `api_version` query parameter. The `openai` Python SDK v1.x supports Azure natively via `AzureOpenAI`.

Chat Completions — Core Structure

PYTHON

```python
from openai import AzureOpenAI
import os

client = AzureOpenAI(
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),   # https://myaoai.openai.azure
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    api_version="2025-04-01-preview"
)

response = client.chat.completions.create(
    model="gpt-4o",           # deployment name, NOT model family name
    messages=[
        {"role": "system", "content": "You are a concise technical assistant."},
        {"role": "user",   "content": "Explain RAG in two sentences."}
    ],
    temperature=0.3,
    max_tokens=512,
    top_p=1.0,
    frequency_penalty=0.0,
    presence_penalty=0.0
)
print(response.choices[0].message.content)
print("Tokens used:", response.usage.total_tokens)
```

Streaming Responses

PYTHON

```python
stream = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Write a haiku about cloud computing."}],
    stream=True
)
for chunk in stream:
    delta = chunk.choices[0].delta
    if delta.content:
        print(delta.content, end="", flush=True)
```

Embeddings

PYTHON

```
response = client.embeddings.create(
    model="text-embedding-3-large",   # deployment name for embedding model
    input="Azure AI Search enables hybrid retrieval at scale."
)
vector = response.data[0].embedding  # list of 3072 floats
print(f"Dimensions: {len(vector)}")
```

Key Parameters

Parameter	Range	Effect
`temperature`	0.0 – 2.0	Randomness; 0 = deterministic, 1+ = creative
`max_tokens`	1 – model limit	Maximum output tokens; input + output must not exceed context window
`top_p`	0.0 – 1.0	Nucleus sampling; alternative to temperature (do not tune both at once)
`frequency_penalty`	-2.0 – 2.0	Penalises token repetition proportional to frequency
`presence_penalty`	-2.0 – 2.0	Penalises any already-used token regardless of count
`response_format`	`{"type":"json_object"}`	Forces valid JSON output; must instruct model in system prompt too
`seed`	Integer	Reproducibility hint (not guaranteed; same seed + temperature=0 for best consistency)
`stream`	bool	Server-sent events for incremental output

✦ **System prompt discipline**
For production use, always include a system prompt that (a) defines the model's persona and constraints, (b) instructs it to cite sources if using RAG, and (c) tells it to respond with "I don't know" rather than hallucinating when context is missing.

Function Calling / Tool Use

Pass a `tools` list of JSON Schema-described functions. The model returns a `tool_calls` array with name + arguments when it decides to call a tool. Your code executes the function and appends the

result as a `tool` role message, then calls the API again to get the final answer.

> **How does token counting work in Azure OpenAI, and why does it matter for cost?**
>
> Every API call is billed on **input tokens + output tokens**. Input tokens include the system prompt, all previous turns in the conversation, and any tool definitions — so long conversations and large system prompts accumulate cost rapidly. Use `tiktoken` to count tokens before sending. For GPT-4o, 1 image counts as a fixed number of tokens depending on resolution (auto-detail uses ~85 base + 170 per 512-px tile). Minimise cost by: (1) summarising conversation history, (2) trimming tool schemas, (3) caching static system prompts with Azure's prompt-caching feature where available.

Azure AI Vision & Computer Vision

Azure AI Vision (Image Analysis 4.0) unifies optical character recognition, object detection, image captioning, smart cropping, and background removal into a single modern API. Custom Vision adds trainable classification and object detection.

IMAGE ANALYSIS 4.0 FEATURES

Caption / Dense Captions: natural-language description per image or per bounding box.

Object Detection: bounding boxes + labels for 10,000+ concepts.

OCR / Read: layout-aware text extraction from images and PDFs; returns word, line, and paragraph bounding polygons.

Smart Crops: generates crop coordinates respecting visual saliency for any target aspect ratio.

Background Removal: segmentation mask returning foreground or background as PNG.

CUSTOM VISION & FACE

Custom Vision (classification): image multi-class and multi-label classifiers trained on as few as 15 images per class.

Custom Vision (detection): object detection with labelled bounding boxes.

Face API (Limited Access): face detection, liveness, verification, and grouping. Requires approval; identification and emotion features restricted per RAI policy.

Spatial Analysis: people-counting and zone analytics for on-premises IoT Edge deployments.

OCR vs. Document Intelligence

Capability	Vision Read (OCR)	Document Intelligence
Primary use	Extract text from natural images and photos	Extract structured fields from business documents
Output	Words, lines, paragraphs with bounding boxes	Key-value pairs, tables, named fields, confidence scores
Prebuilt models	No	Yes (Invoice, Receipt, W-2, ID, Contract, Tax…)
Custom training	No	Yes (Neural, Template, Composed models)
Best for	Street signs, screenshots, handwriting in photos	Invoice automation, claims processing, KYC

> **API version matters**
> Image Analysis 4.0 uses the `/imageanalysis:analyze` endpoint and features GA'd in 2024. Older `/analyze` and `/read` endpoints are Image Analysis 3.x and will be retired — migrate to 4.0 for new projects.

Azure AI Language Services

Azure AI Language consolidates a broad suite of NLP capabilities — sentiment analysis, NER, key phrase extraction, summarisation, conversational understanding, custom text classification, and question answering — under one unified SDK and resource.

Available Capabilities

SENTIMENT & OPINION MINING

Document-level and sentence-level sentiment (positive / neutral / negative) with aspect-opinion pairs. Supports 120+ languages.

NAMED ENTITY RECOGNITION

Identifies persons, locations, organisations, dates, phone numbers, URLs, IP addresses, and domain-specific entities (healthcare, finance) out of the box.

PII DETECTION & REDACTION

Detects and optionally masks email addresses, SSN, credit card numbers, phone numbers, and healthcare entities in text and documents.

SUMMARISATION

Extractive (selects representative sentences) and abstractive (generates a new summary). Conversation summarisation available for call centre transcripts.

CLU (CONVERSATIONAL LANGUAGE UNDERSTANDING)

Replacement for LUIS. Train intent classification and entity extraction models on your own utterances. Integrates with Bot Framework and Copilot Studio.

CUSTOM QUESTION ANSWERING

Replacement for QnA Maker. Build FAQ knowledge bases from documents, URLs, or manual Q&A pairs; returns ranked answers with confidence scores.

PYTHON

```python
from azure.ai.textanalytics import TextAnalyticsClient
from azure.core.credentials import AzureKeyCredential
import os

client = TextAnalyticsClient(
    endpoint=os.getenv("LANGUAGE_ENDPOINT"),
    credential=AzureKeyCredential(os.getenv("LANGUAGE_KEY"))
)

docs = [
    "Azure AI Search delivers blazing-fast retrieval for enterprise knowledge bases.",
    "The deployment failed at 2:00 AM and the team was furious."
]

# Sentiment + opinion mining
results = client.analyze_sentiment(docs, show_opinion_mining=True)
for doc in results:
    print(f"Overall: {doc.sentiment}")
    for sentence in doc.sentences:
        print(f"  Sentence sentiment: {sentence.sentiment}")
        for opinion in sentence.mined_opinions:
            print(f"    Aspect '{opinion.target.text}' is {opinion.target.sentiment}")
```

Azure AI Speech Services

Azure AI Speech provides enterprise-grade speech recognition, synthesis, translation, and speaker recognition, with support for custom acoustic and language models tailored to domain-specific vocabulary.

SPEECH-TO-TEXT (STT)

Real-time: streaming recognition from microphone or audio stream with word-level timestamps and confidence scores.

Batch: asynchronous transcription of audio files in blob storage; returns JSON with detailed timestamps.

Fast Transcription: low-latency file transcription (faster than real-time batch) for short audio.

Whisper on Azure: OpenAI Whisper model available as an Azure OpenAI deployment for multilingual transcription.

TEXT-TO-SPEECH (TTS)

Neural voices: 500+ prebuilt neural voices across 140+ locales. SSML supported for pitch, rate, emphasis, and pauses.

Custom Neural Voice: fine-tune a voice from your own recorded speech samples (requires Microsoft approval).

Speech Translation: real-time speech-to-speech or speech-to-text translation across 100+ language pairs.

Speaker Recognition: speaker verification (is this the same person?) and speaker identification (who is speaking?).

Python SDK Quick Start

PYTHON

```python
import azure.cognitiveservices.speech as speechsdk
import os

speech_config = speechsdk.SpeechConfig(
    subscription=os.getenv("SPEECH_KEY"),
    region=os.getenv("SPEECH_REGION")
)
speech_config.speech_recognition_language = "en-US"

# Microphone input
audio_config = speechsdk.audio.AudioConfig(use_default_microphone=True)
recognizer = speechsdk.SpeechRecognizer(
    speech_config=speech_config,
    audio_config=audio_config
)
result = recognizer.recognize_once_async().get()
if result.reason == speechsdk.ResultReason.RecognizedSpeech:
    print("Recognised:", result.text)
```

Custom Speech — when and how

Azure AI Document Intelligence

Azure AI Document Intelligence (formerly Form Recognizer) extracts structured data — text, tables, key-value pairs, and typed fields — from PDFs, images, and Office documents using AI-powered layout analysis and prebuilt or custom models.

Prebuilt Models

Model ID	Extracts
`prebuilt-layout`	Text, tables, figures, and key-value pairs from any document (the base model used internally by all others)
`prebuilt-invoice`	Vendor, customer, line items, amounts, dates, PO numbers
`prebuilt-receipt`	Merchant, transaction date, items, subtotals, tax, total
`prebuilt-idDocument`	Passport, national ID — name, DOB, document number, expiry
`prebuilt-contract`	Parties, term, jurisdiction, payment terms
`prebuilt-tax.us.w2`	US W-2 wage and tax statement fields
`prebuilt-read`	Text-only extraction with language detection; fastest and cheapest model

Python SDK

PYTHON

```python
from azure.ai.documentintelligence import DocumentIntelligenceClient
from azure.core.credentials import AzureKeyCredential
import os

di_client = DocumentIntelligenceClient(
    endpoint=os.getenv("DI_ENDPOINT"),
    credential=AzureKeyCredential(os.getenv("DI_KEY"))
)

with open("invoice.pdf", "rb") as f:
    poller = di_client.begin_analyze_document("prebuilt-invoice", f)

result = poller.result()
for invoice in result.documents:
    fields = invoice.fields
    vendor = fields.get("VendorName")
    total  = fields.get("InvoiceTotal")
    if vendor:
        print("Vendor:", vendor.content, "  Confidence:", vendor.confidence)
    if total:
        print("Total: ", total.content, "  Confidence:", total.confidence)
```

✦ **Custom models**

When prebuilt models miss fields specific to your forms, train a custom model. Neural models handle variable layouts (recommended). Template models are faster but require consistent form structure. Compose up to 100 models for a single endpoint that auto-selects the right sub-model by document type.

Azure AI Search — Vector & Hybrid Search

Azure AI Search is a fully managed enterprise search service that combines BM25 keyword retrieval, approximate nearest-neighbour (ANN) vector search, and a neural semantic ranker into a unified hybrid pipeline. It is the retrieval backbone for most enterprise RAG architectures on Azure.

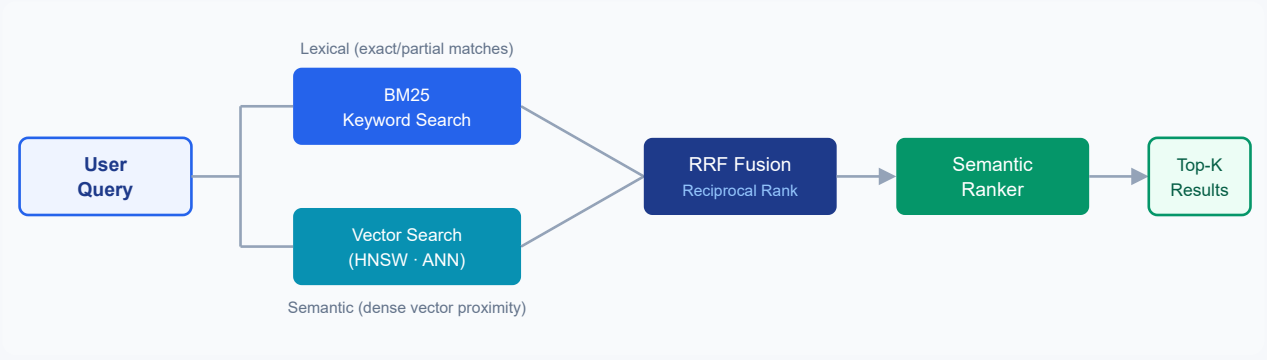


Figure 3 — Azure AI Search hybrid retrieval pipeline. BM25 and vector search run in parallel; scores are merged by Reciprocal Rank Fusion (RRF); the optional semantic ranker applies a cross-encoder for final re-ranking.

Index Schema Essentials

Field Attribute	Meaning
`searchable`	Field participates in BM25 full-text search; triggers inverted index creation
`filterable`	Field can be used in OData `$filter` expressions
`sortable`	Field can be used in `$orderby`
`facetable`	Field can be used for faceted navigation counts
`retrievable`	Field is returned in search results
`key`	Unique document identifier (required, exactly one per index)
`dimensions`	Vector field: number of dimensions (must match your embedding model)
`vectorSearchProfile`	Vector field: references an HNSW or exhaustive KNN algorithm config

Python: Hybrid Search Query

PYTHON

```python
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery
from azure.core.credentials import AzureKeyCredential
import os

search_client = SearchClient(
    endpoint=os.getenv("SEARCH_ENDPOINT"),
    index_name="documents",
    credential=AzureKeyCredential(os.getenv("SEARCH_ADMIN_KEY"))
)

def hybrid_search(user_query: str, query_vector: list, top: int = 5):
    vector_query = VectorizedQuery(
        vector=query_vector,
        k_nearest_neighbors=50,    # retrieve 50 vector candidates, then fuse
        fields="content_vector"
    )
    results = search_client.search(
        search_text=user_query,           # BM25 keyword leg
        vector_queries=[vector_query],    # vector leg
        select=["id", "title", "content"],
        query_type="semantic",            # enable semantic ranker
        semantic_configuration_name="my-semantic-config",
        top=top
    )
    return [dict(r) for r in results]
```

> **Semantic ranker**
>
> The semantic ranker is a cross-encoder that re-reads the query against each candidate passage. It adds ~100–200 ms latency but significantly improves relevance for question-like queries. It is billed separately (per 1,000 semantic queries). Available on Basic tier and above.

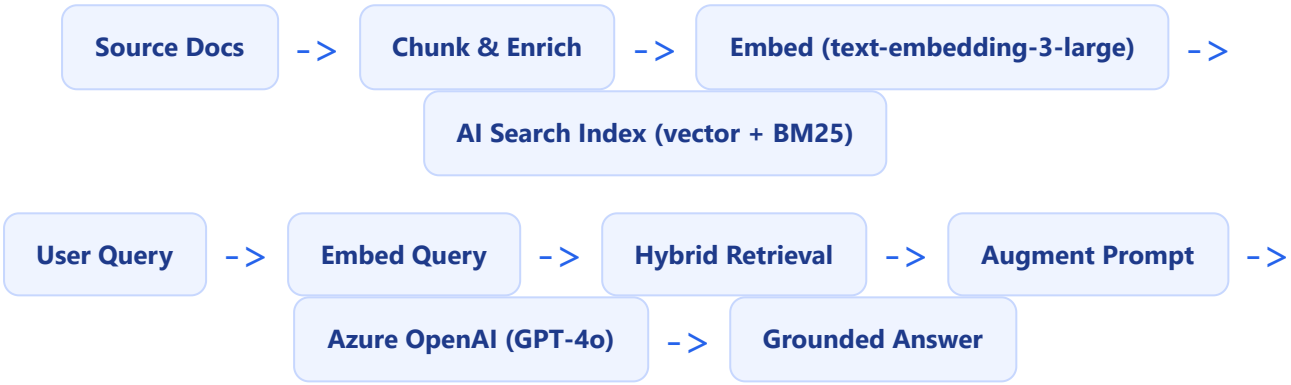
What is Reciprocal Rank Fusion (RRF) and why does Azure AI Search use it for hybrid retrieval?

RRF is a score-merging algorithm: it ranks documents by the sum of `1 / (k + rank)` across each retrieval leg, where *k* (typically 60) dampens the effect of early rankings. Because it uses rank positions rather than raw scores, it correctly merges BM25 scores (whose scale depends on term frequencies) with cosine-similarity vector scores (bounded 0–1) without rescaling. The result is that a document appearing in the top ranks of *both* legs is pushed to the top of the fused result, regardless of absolute score values.

RAG Pattern End-to-End

Retrieval-Augmented Generation (RAG) grounds an LLM's answers in a private knowledge corpus, eliminating hallucinations about proprietary data. The canonical Azure RAG stack is: **Azure Blob / SharePoint → Document Intelligence → Azure AI Search (hybrid) → Azure OpenAI (GPT-4o)**.

Two-Phase Architecture



Complete RAG Function

PYTHON

```python
from openai import AzureOpenAI
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery
from azure.core.credentials import AzureKeyCredential
import os

aoai = AzureOpenAI(
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    api_version="2025-04-01-preview"
)
search = SearchClient(
    os.getenv("SEARCH_ENDPOINT"), "documents",
    AzureKeyCredential(os.getenv("SEARCH_ADMIN_KEY"))
)

def get_embedding(text: str) -> list:
    return aoai.embeddings.create(
        model="text-embedding-3-large", input=text
    ).data[0].embedding

def answer_with_rag(question: str) -> str:
    # 1. Embed the question
    q_vec = get_embedding(question)

    # 2. Hybrid retrieval
    vq = VectorizedQuery(vector=q_vec, k_nearest_neighbors=30, fields="content_vector"
    hits = search.search(
        search_text=question,
        vector_queries=[vq],
        select=["title", "content"],
        top=4
    )
    context = "\n\n".join(
        f"[Source: {h['title']}]\n{h['content']}" for h in hits
    )

    # 3. Augmented generation
    resp = aoai.chat.completions.create(
        model="gpt-4o",
        messages=[
            {"role": "system",
             "content": (
                 "Answer ONLY from the context below. "
                 "Cite sources as [Source: title]. "
                 "If the answer is not in the context, say 'I don't know'.\n\n"
                 + context
             )},
```

```
            {"role": "user", "content": question}
        ],
        temperature=0.0,
        max_tokens=800
    )
    return resp.choices[0].message.content
```

Chunking Strategies

Strategy	Description	When to Use
Fixed-size	Split by character count (e.g. 512 chars, 20 % overlap)	Uniform text; quick to implement
Sentence-based	Split on sentence boundaries using NLTK / spaCy	When preserving sentence meaning matters
Recursive	Try paragraph → sentence → word splits in order	Mixed document types; default LangChain strategy
Semantic	Split where embedding cosine similarity drops (Azure AI Search "text split" skill)	Best relevance; more compute cost
Document-aware	Use Document Intelligence layout to chunk at table / section boundaries	PDFs, invoices, reports with rich structure

> ⚠ **Hallucination guard**
> Always instruct the model to answer from context only and explicitly say "I don't know" when the context does not contain the answer. Evaluate with Azure AI Foundry's built-in groundedness metric (requires GPT-4 as judge) to catch answer-context drift before releasing to production.

> **What is the difference between semantic ranking and vector search in Azure AI Search?**
>
> **Vector search** finds documents whose stored embeddings are geometrically close (high cosine similarity) to the query embedding. It captures semantic meaning but requires documents to have been pre-embedded and indexed in vector fields. **Semantic ranking** is a separate, post-retrieval step: it uses a cross-encoder transformer to re-read the query against each candidate passage and re-rank them by passage relevance. Semantic ranking can be applied to the output of keyword, vector, or hybrid search — it does not require vector fields. The two are complementary: vector search provides good candidate recall; semantic ranking improves precision of the final top-K.

How do you prevent a RAG system from returning outdated information after documents are updated?

Use **incremental indexing** via an AI Search Indexer with a `dataChangeDetectionPolicy` (e.g., SQL HighWaterMark or Blob Storage last-modified timestamp). The indexer re-crawls only changed or new documents and updates/deletes the corresponding index entries. For near-real-time needs, call the index REST API or SDK directly from your document management pipeline (push model). Always store a `last_updated` metadata field in the index so you can filter with `$filter=last_updated ge 2025-01-01T00:00:00Z` or display freshness to users.

Azure AI Foundry & ML Studio

Azure AI Foundry (released 2024, unifying Azure Machine Learning Studio and Azure AI Studio) is the end-to-end platform for building, evaluating, and deploying AI applications and models on Azure. It introduces a **Hub** → **Project** organisational model shared by all team members.

HUB & PROJECT MODEL

A **Hub** is the top-level Azure resource that holds shared infrastructure: compute, storage, connections to services (Azure OpenAI, AI Search, Application Insights), and network/identity configuration.

A **Project** lives inside a Hub and isolates a single application or team workstream. Projects share the Hub's connections and compute but have separate experiment tracking, deployments, and file stores.

MODEL CATALOG

Browse and deploy 1,600+ models: Azure OpenAI models, Meta Llama 3/4, Mistral Large, Phi-4, Cohere Command R+, Falcon, and many more via **serverless API deployments** (pay-per-token, no GPU provisioning) or **managed online endpoints** (dedicated VMSS).

Each model card includes benchmark scores (MMLU, HumanEval, MT-Bench) and pricing to aid model selection.

PROMPT FLOW

A visual DAG-based orchestration framework. Nodes can be: **LLM** (call any deployed model), **Prompt** (Jinja2 template), **Python** (arbitrary code), **Embedding**, and **Vector DB lookup**. Flows are versioned, testable with batch inputs, and deployable to managed endpoints.

EVALUATIONS

Built-in metrics: **Groundedness** (is the answer supported by context?), **Relevance** (does it answer the question?), **Coherence**, **Fluency**, and **Similarity**. All use GPT-4 as an LLM judge. Results feed an experiment tracker for A/B comparison of prompt variants and model choices. Custom evaluators (Python functions) are also supported.

Azure ML Studio vs AI Foundry — what moved where

Responsible AI & Content Safety

Microsoft's Responsible AI Standard applies to all Azure AI Services. Azure OpenAI additionally implements runtime content filters that inspect every request and response — and these filters are mandatory; they cannot be disabled unless Microsoft grants an exception via the Limited Access program.

Microsoft's Six Responsible AI Principles

FAIRNESS

AI systems should treat all people equitably and not produce discriminatory outcomes based on race, gender, age, disability, nationality, religion, or other protected characteristics.

RELIABILITY & SAFETY

Systems should perform as designed, handle failures gracefully, and not cause unintended harm. This includes testing under adversarial conditions and setting safe fallback behaviors.

PRIVACY & SECURITY

Data used to interact with AI must be protected. Azure AI Services do not use customer prompts or data to retrain models. Encryption at rest and in transit is enforced.

INCLUSIVENESS

AI should empower people of all abilities and backgrounds. Speech services support diverse accents; Vision APIs are tested across skin tones; Language APIs cover 100+ languages.

TRANSPARENCY

Users and operators should understand what AI systems can and cannot do. Model cards, system cards, and capability documentation accompany all Azure OpenAI model releases.

ACCOUNTABILITY

People must be accountable for AI systems they create and deploy. Azure provides audit logs, access reviews, and a human oversight framework (HITL) recommendation for high-stakes scenarios.

Azure OpenAI Content Filters

Harm Category	Severity Levels	Default Action
Hate & Fairness	Safe / Low / Medium / High (0–6)	Block at Medium and above (configurable)
Violence	Safe / Low / Medium / High	Block at Medium and above
Sexual	Safe / Low / Medium / High	Block at Low and above (stricter default)
Self-harm	Safe / Low / Medium / High	Block at Medium and above
Jailbreak	Detected / Not detected	Always block when detected (Prompt Shields)
Protected material (text)	Detected / Not detected	Block or annotate depending on policy
Groundedness (RAG)	Grounded / Ungrounded	Annotate; block configurable

⚠ **Limited Access features**

Several Azure AI features require explicit Microsoft approval before use: Face API identification/emotion, Custom Neural Voice, Content Safety severity override, and Azure OpenAI access itself (though generally available, it still requires an access request form in some regions). Check the Limited Access policy before designing your architecture.

Can Azure OpenAI content filters be turned off for a production application?

> No, not by default. Content filters run on every request and response and cannot be disabled. If a business scenario legitimately requires different thresholds (e.g., a medical platform discussing self-harm with patients), you can submit a **modified content filter configuration request** to Microsoft. Approval is case-by-case, time-limited, and still subject to the usage policies. Even with an approved custom configuration you cannot fully disable all filters — you can only relax specific severity thresholds for specific harm categories in specific deployment contexts.

Authentication: Keys, Endpoints & RBAC

Azure AI Services support two authentication modes: API key (fast and simple) and Microsoft Entra ID (recommended for production). Entra ID authentication integrates with Managed Identities so no secret is ever stored in code or configuration files.

Authentication Cheat Sheet

Method	Header / Mechanism	When to Use
API Key	`api-key: <key>` (AOAI) or `Ocp-Apim-Subscription-Key: <key>`	Development, quick tests, CI pipelines with secret managers
Entra ID (MSAL/SDK)	Bearer token in `Authorization` header; refreshed automatically by SDK	Production apps; eliminates secret rotation risk
Managed Identity (System)	Identity assigned to the Azure resource (VM, App Service, Function, AKS)	Server-to-server calls from Azure-hosted workloads
Managed Identity (User)	Explicit identity resource shared across multiple Azure resources	When multiple services need same permissions; easier role assignment
Workload Identity (AKS)	Service account token federated with Entra ID	Kubernetes-hosted apps; replaces in-cluster secrets

Key RBAC Roles

Role	Permissions
Cognitive Services User	Call APIs (read/write); cannot manage resources or see keys
Cognitive Services Contributor	Full control including reading keys and creating resources
Cognitive Services OpenAI User	Call Azure OpenAI APIs only (subset of Cognitive Services User)
Cognitive Services OpenAI Contributor	Create deployments and call APIs; cannot read subscription keys
Search Index Data Reader	Execute queries on a specific AI Search index
Search Index Data Contributor	Read + write index documents (ingest, update, delete)

Managed Identity Pattern

PYTHON

```python
from azure.identity import DefaultAzureCredential
from azure.identity import get_bearer_token_provider
from openai import AzureOpenAI
import os

# DefaultAzureCredential tries: environment vars → Managed Identity → CLI → VS Code
credential = DefaultAzureCredential()

token_provider = get_bearer_token_provider(
    credential,
    "https://cognitiveservices.azure.com/.default"
)

client = AzureOpenAI(
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    azure_ad_token_provider=token_provider,  # no api_key needed
    api_version="2025-04-01-preview"
)

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Hello from Managed Identity!"}]
)
print(response.choices[0].message.content)
```

Key rotation

Every Azure AI Services resource has two API keys. Rotate without downtime by: (1) updating your app to the secondary key, (2) regenerating the primary key in the portal, (3) updating your app back to the regenerated primary. Automate this with Azure Key Vault + Event Grid-triggered Functions. With Managed Identity you skip this entirely.

What is the difference between Cognitive Services User and Cognitive Services OpenAI User RBAC roles?

Cognitive Services User grants access to all Azure AI Services APIs under a resource — Vision, Language, Speech, Document Intelligence, and Azure OpenAI. **Cognitive Services OpenAI User** is scoped specifically to the Azure OpenAI API surface and cannot call non-OpenAI services. Use the OpenAI-scoped role to follow the principle of least privilege when a workload only needs to call GPT/embedding endpoints. Neither role allows listing API keys (that requires Contributor) — both rely on Entra ID token auth when used for production least-privilege access.

Python SDK Integration Patterns

Each Azure AI service has a dedicated Python SDK package under the `azure-ai-*` namespace, plus the `openai` package with native Azure support. Use `azure-identity` for credential management across all services.

Package Reference

Service	Package	Key Client Class
Azure OpenAI	`openai`	`AzureOpenAI`
Azure AI Language	`azure-ai-textanalytics`	`TextAnalyticsClient`
Azure AI Language (CLU)	`azure-ai-language-conversations`	`ConversationAnalysisClient`
Azure AI Speech	`azure-cognitiveservices-speech`	`SpeechRecognizer`, `SpeechSynthesizer`
Azure AI Vision	`azure-ai-vision-imageanalysis`	`ImageAnalysisClient`
Azure AI Document Intelligence	`azure-ai-documentintelligence`	`DocumentIntelligenceClient`
Azure AI Search	`azure-search-documents`	`SearchClient`, `SearchIndexClient`
Azure AI Projects (Foundry)	`azure-ai-projects`	`AIProjectClient`
Identity	`azure-identity`	`DefaultAzureCredential`, `ClientSecretCredential`

Environment Variables Convention

BASH

```
# Azure OpenAI
AZURE_OPENAI_ENDPOINT=https://myaoai.openai.azure.com/
AZURE_OPENAI_API_KEY=<key>
# AZURE_OPENAI_API_VERSION is read automatically by some versions of SDK

# Azure AI Language
LANGUAGE_ENDPOINT=https://my-language.cognitiveservices.azure.com/
LANGUAGE_KEY=<key>

# Azure AI Speech
SPEECH_KEY=<key>
SPEECH_REGION=eastus

# Azure AI Document Intelligence
DI_ENDPOINT=https://my-di.cognitiveservices.azure.com/
DI_KEY=<key>

# Azure AI Search
SEARCH_ENDPOINT=https://my-search.search.windows.net
SEARCH_ADMIN_KEY=<admin-key>
SEARCH_QUERY_KEY=<query-only-key>
```

Async Pattern

PYTHON

```python
import asyncio
from openai import AsyncAzureOpenAI

async def ask(question: str) -> str:
    async with AsyncAzureOpenAI(
        azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
        api_key=os.getenv("AZURE_OPENAI_API_KEY"),
        api_version="2025-04-01-preview"
    ) as client:
        resp = await client.chat.completions.create(
            model="gpt-4o",
            messages=[{"role": "user", "content": question}]
        )
        return resp.choices[0].message.content

# Fan-out multiple questions concurrently
questions = ["What is HNSW?", "What is RRF?", "What is PTU?"]
answers = asyncio.run(asyncio.gather(*[ask(q) for q in questions]))
```

✦ **Retry policy**

All `azure-*` SDK clients include automatic retry with exponential back-off for transient HTTP 429 (rate limit) and 5xx errors. Tune with `RetryPolicy(total=5, backoff_factor=0.8)` passed to the client's `retry_policy` constructor argument. For Azure OpenAI, the `openai` SDK raises `RateLimitError` — wrap calls in a `tenacity` retry decorator for production resilience.

> **How do you securely provide Azure credentials to a Python application running in Azure App Service?**
>
> Enable the **System-Assigned Managed Identity** on the App Service in the Azure Portal (Identity → Status: On). Then grant that identity the appropriate RBAC role on each target resource (e.g., *Cognitive Services OpenAI User* on the Azure OpenAI resource). In code, use `DefaultAzureCredential()` — it automatically uses the Managed Identity token when running in Azure. Locally, the same code falls through to Azure CLI credentials (`az login`). This pattern requires zero secrets in environment variables, no key rotation, and works transparently across environments.

Cost, Quotas & Azure OpenAI vs. Direct OpenAI API

Understanding Azure OpenAI pricing and quota mechanics is essential for production planning and a common interview topic. The billing model differs significantly from the direct OpenAI API in both structure and commercial terms.

Pricing Models

Model	Unit	Billing Scope
Standard deployment (chat/completion)	Per 1,000 tokens (input priced lower than output)	Per deployment per region
Embedding models	Per 1,000 tokens (input only)	Per deployment
Provisioned Managed (PTU)	Per PTU-hour	1-month or 1-year commitment (discounted)
Batch API	~50 % discount vs Standard	Per token; 24-hour completion SLA
DALL-E 3	Per image (varies by size and quality)	Per generation request
Whisper	Per minute of audio	Per transcription minute
Vision (image tokens)	Fixed + tile-based tokens per image	Included in chat token billing

Quota Concepts

Quota Type	Description
TPM (Tokens Per Minute)	Rate limit on total tokens (input + output) processed per minute across all calls to a deployment
RPM (Requests Per Minute)	Rate limit on number of API calls; usually derived from TPM (TPM / 6 = approximate RPM)
Regional quota	Total TPM assigned to your subscription per region; shared across all deployments in that region
Model quota	Some models have separate per-subscription caps independent of regional quota

> **Quota increase**
> Request increases via the Azure Portal: Azure OpenAI resource → Deployments → select deployment → Request quota increase. Processing takes 3–5 business days. For very large scale, Provisioned Managed (PTU) deployments bypass TPM limits entirely — their throughput is reserved at provisioning time.

Azure OpenAI vs. Direct OpenAI API

Dimension	Azure OpenAI	Direct OpenAI API (api.openai.com)
Data residency	Stays in your chosen Azure region; never leaves tenant	Processed in OpenAI's US/EU datacentres
Training on your data	No — Microsoft contractually commits to this	No (opt-out by default for API users)
Auth / access control	Azure RBAC + Entra ID + Managed Identity + Private Link	API key only; org-level only
Network isolation	VNet integration, Private Endpoints, IP allowlist	Public internet only
Content filtering	Mandatory; configurable thresholds; Prompt Shields	OpenAI Moderation API (optional, separate call)
SLA	99.9 % availability SLA (Standard); PTU has separate SLA	No formal SLA
Compliance	ISO 27001, SOC 2, HIPAA, EU Data Boundary, FedRAMP (select regions)	SOC 2, limited certifications
Model availability	Slightly behind Direct OpenAI for newest releases; regional capacity may vary	First access to new models
Billing	Azure subscription invoice; EA/MCA pricing, Azure credits	OpenAI credit-card / invoice billing
Rate limits	Per-deployment TPM; increase via quota request	Per-org tier; increase by upgrading plan

✓ **When to choose Azure OpenAI**

Choose Azure OpenAI if your organisation: (1) has strict data sovereignty or compliance requirements, (2) already uses Azure and wants unified billing/RBAC, (3) needs VNet/Private Link integration, or (4) is subject to regulated industry requirements (HIPAA, FedRAMP). For a personal side project or early-stage startup without compliance requirements, the direct OpenAI API is simpler and gives faster access to new models.

You are getting HTTP 429 errors from Azure OpenAI. What are your options?

A 429 means you hit a quota limit (TPM or RPM). Options in order of effort: (1) **Retry with exponential back-off** — transient spikes often resolve themselves; the SDK does this automatically. (2) **Reduce request rate** — add throttling logic or a queue in front of the API. (3) **Request a quota increase** in the Azure Portal (takes 3–5 days). (4) **Use multiple deployments / regions** — distribute load across deployments in different regions, each with its own quota. (5) **Switch to Provisioned Managed (PTU)** — PTU deployments do not have TPM rate limits; their throughput is reserved. (6) **Use the Batch API** for non-time-sensitive workloads — async with 50 % price reduction and no TPM limit.

How does Azure OpenAI handle data privacy differently from the direct OpenAI API?

With Azure OpenAI, Microsoft contractually guarantees that: (a) your prompts, completions, and fine-tuning data are **not used to train or improve OpenAI's or Microsoft's models**, (b) data stays within your **Azure region** (with Global Standard routing as the exception — disabling it keeps data in-region), and (c) the service is subject to your Azure DPA (Data Processing Agreement). Access to data in motion and at rest requires your tenant's keys. The direct OpenAI API also does not train on API data by default, but it lacks the enterprise legal guarantees, VNet isolation, and Azure compliance certifications. For regulated workloads, Azure OpenAI is the required choice.

Best Practices & Interview Cheat Sheet

A consolidated reference of key patterns, pitfalls, and metrics for production deployments and technical interviews.

✦ **Production checklist**

Use Managed Identity instead of API keys in all Azure-hosted workloads. Store any keys that must be used in Azure Key Vault with automatic rotation. Enable diagnostic logging on all AI resources and route to Log Analytics. Set up alerts on HTTP 429 rate, latency P99, and token consumption per deployment. For Azure OpenAI, configure a custom content filter policy and review Content Safety block logs weekly.

⚠ **Common pitfalls**

(1) Calling the model by its catalog name instead of the deployment name — always use the deployment name in the `model` field. (2) Ignoring system prompt token cost — a 2,000-token system prompt is included in every request. (3) Not validating chunks before embedding — empty or near-empty chunks produce noisy embeddings that pollute retrieval. (4) Using the wrong API version — Azure OpenAI feature availability varies by `api_version`; always pin to a stable version in production. (5) Not handling semantic ranker availability — it requires Basic+ tier; code should degrade gracefully.

Key Numbers to Remember

Item	Value
GPT-4o context window	128,000 tokens
GPT-4.1 context window	1,000,000 tokens
o4-mini context window	200,000 tokens
text-embedding-3-large dimensions	3,072 (can be truncated to 256 / 512 / 1,024 / 1,536)
text-embedding-3-small dimensions	1,536
ada-002 dimensions (legacy)	1,536
DALL-E 3 max sizes	1024×1024, 1024×1792, 1792×1024
Whisper max file size	25 MB per file
Document Intelligence max file size	500 MB; 2,000 pages
AI Search Free tier limit	3 indexes, 50 MB storage, 10,000 docs
Semantic ranker (AI Search) default	Re-ranks top 50 candidates to return top K
RRF k constant	60 (default in Azure AI Search)
Content filter severity levels	0 (safe), 2 (low), 4 (medium), 6 (high)

Architecture Decision Map

Scenario	Recommended Service(s)
Chat over private documents	Azure AI Search (hybrid) + Azure OpenAI (GPT-4o) — RAG pattern
Automate invoice data extraction	Azure AI Document Intelligence — prebuilt-invoice
Moderate user-generated content	Azure AI Content Safety (harm categories + custom blocklists)
Transcribe customer call recordings	Azure AI Speech — Batch Transcription or Whisper deployment
Extract intent from chat messages	Azure AI Language — CLU (Conversational Language Understanding)
Semantic search across product catalogue	Azure AI Search — vector search with text-embedding-3-large
Fine-tune a model on proprietary data	Azure OpenAI fine-tuning (GPT-4o-mini / GPT-4o, JSONL training format)
Run any open-source model (Llama, Phi)	Azure AI Foundry — Model Catalog serverless endpoint or managed online endpoint
OCR on photos / low-quality scans	Azure AI Vision — Image Analysis 4.0 Read API
Real-time voice assistant	Azure AI Speech STT + Azure OpenAI + TTS (pipeline or GPT-4o Realtime API)